

Table of contents

Part 4 — Pretty URLs: a reverse proxy + local DNS	1
How this works: two jobs, two tools	1
Step 1 — Put the services on a shared Docker network	2
Step 2 — Free port 80 and extend Pi-hole onto the tailnet	2
Step 3 — Deploy Caddy	4
Step 4 — Add the <code>.home</code> names to Pi-hole's DNS	5
Step 5 — Make Pi-hole your tailnet's DNS	5
Step 6 — Try it	6
The pattern you'll reuse	6
Caveats	6
Recap	7

Part 4 — Pretty URLs: a reverse proxy + local DNS

The payoff of this part: type `http://pihole.home` or `http://abs.home` in any browser on your tailnet — at home or on cellular — and land on the right service, no port numbers, no IP addresses. And a pattern you'll reuse for every service you add from here on.

Right now you reach your two services awkwardly: `http://192.168.1.50/admin` for Pi-hole, `http://homelab:13378` for Audiobookshelf — an IP here, a port there. This part introduces the machinery that gives **every** service one clean name, starting with these two. When we add the dashboard in [Part 5](#), it'll slot into the same system in three lines.

! Why not `pihole.com`?

A `.com` (or any public) name belongs to whoever registered it on the internet — you can't point `pihole.com` at your Pi any more than you can point `google.com` at it. We use a **private** suffix, `.home`, that exists only on *your* network.

How this works: two jobs, two tools

A working URL needs two separate things:

1. **The name has to resolve to your Pi.** `pihole.home` must turn into the Pi's address — that's **DNS** (we add records to Pi-hole and let Tailscale carry them to every device).
2. **Traffic hitting the Pi on port 80 has to reach the right service.** `pihole.home` and `abs.home` both arrive on port 80, but Pi-hole and Audiobookshelf listen on *different* ports. Something must read the requested name and forward to the correct backend — a **reverse proxy** (we'll use **Caddy**, the simplest to configure).

<code>http://pihole.home</code>	DNS	the Pi (port 80)	Caddy routes by name	<code>pihole:80</code>
<code>http://abs.home</code>	DNS	the Pi (port 80)	Caddy routes by name	<code>audiobookshelf:80</code>

Step 1 — Put the services on a shared Docker network

Caddy reaches each backend by **container name** (pihole, audiobookshelf), which only works if they share a Docker network. Create it:

```
docker network create homelab
```

Now attach **Audiobookshelf** to it. (Audiobookshelf runs from a plain `docker run` in Part 2, so we attach it imperatively here. Pi-hole is a Compose project, so we'll attach it the *declarative* way in Step 2 — read the callout below for why that distinction matters.)

```
docker network connect homelab audiobookshelf
```

Any container on the `homelab` network can now reach the others by name — no IP addresses needed between containers.

 **Attach Compose services in the Compose file, not with `docker network connect`**

It's tempting to run `docker network connect homelab pihole` here too — but it won't survive. The very next step recreates the Pi-hole container (`docker compose up --force-recreate`), and a **recreate rebuilds the container with only the networks declared in its `compose.yaml`** — any network you attached by hand is silently dropped. Caddy would then fail to resolve `pihole:80`. So for Pi-hole (and every Compose service) we add the `homelab` network *inside* the Compose file in Step 2, where a `recreate` always re-attaches it. The imperative `docker network connect` is only for Audiobookshelf because it isn't a Compose service. (If you ever `docker rm` and re-run Audiobookshelf — e.g. the update in [Part 2's Maintenance](#) — re-run `docker network connect homelab audiobookshelf` afterward for the same reason.)

Step 2 — Free port 80 and extend Pi-hole onto the tailnet

Three changes to Pi-hole's `~/pihole/compose.yaml`, all needed before the proxy works:

- **Free host port 80.** Caddy will be the single front door on port 80, so move Pi-hole's web UI to 8081.
- **Listen on the tailnet, not just the LAN.** In Part 3 you bound Pi-hole to your LAN IP because it was a home-only service. To make `.home` names (and Pi-hole's own ad-blocking) work *everywhere*, Pi-hole's DNS must also answer on the Pi's Tailscale interface — so drop the `PIHOLE_IP` prefix and let it listen on all interfaces.
- **Join the homelab network declaratively** (per the Step 1 callout) so Caddy can reach it as `pihole:80`, and so the `recreate` below — and every future `docker compose up` — re-attaches it automatically.

Replace `~/pihole/compose.yaml` with this updated version (the `ports:` and new `networks:` blocks are the only changes from Part 3):

```

cat > ~/pihole/compose.yaml <<'EOF'
services:
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "53:53/tcp"          # all interfaces (LAN + tailnet); was
${PIHOLE_IP}:53:53
      - "53:53/udp"
      - "8081:80/tcp"        # web UI moved off 80, freeing it for Caddy;
was ${PIHOLE_IP}:80:80

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"

    volumes:
      - ./etc-pihole:/etc/pihole

    networks:
      - homelab              # NEW: declarative attachment, survives every
recreate

    restart: unless-stopped

networks:
  homelab:
    external: true          # NEW: the shared network you created in Step 1
EOF

```

Then recreate the container so the new ports and network take effect:

```
cd ~/pihole && docker compose up -d --force-recreate
```

i Is listening on all interfaces safe?

Yes, in this setup. The only networks that can reach the Pi are your home LAN (behind the router) and your tailnet (authenticated WireGuard). The one unbreakable rule, same as Part 3: **never port-forward port 53 (or 80) from your router**. As long as you don't, "all interfaces" just means "my LAN and my tailnet," which is exactly who should be able to use it.

Pi-hole's admin UI now lives at <http://192.168.1.50:8081/admin> (your Pi's LAN IP) — but in

a moment you'll reach it as `http://pihole.home` instead.

Step 3 — Deploy Caddy

Caddy is its own small stack:

```
mkdir -p ~/proxy && cd ~/proxy
```

Create `~/proxy/Caddyfile` — the entire routing table:

```
cat > Caddyfile <<'EOF'
# The http:// prefix tells Caddy to serve plain HTTP and NOT try to fetch a
# TLS certificate (there's no public CA for a private .home name).

http://pihole.home {
    redir / /admin 302          # land on the admin page directly
    reverse_proxy pihole:80
}

http://abs.home {
    reverse_proxy audiobookshelf:80
}
EOF
```

Create `~/proxy/compose.yaml`:

```
cat > compose.yaml <<'EOF'
services:
  caddy:
    container_name: caddy
    image: caddy:latest
    ports:
      - "80:80"                # the single front door, on all interfaces
      (LAN + tailnet)
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - homelab
    restart: unless-stopped

networks:
  homelab:
    external: true            # the shared network from Step 1
```

```
volumes:
  caddy_data:
  caddy_config:
EOF
```

Start it:

```
docker compose up -d
docker compose logs --tail 20
```

Caddy listens on `0.0.0.0:80` — all interfaces, including the Pi’s Tailscale `100.x.y.z` — and reaches each backend by container name over the `homelab` network, so it ignores host port mappings entirely. That’s why moving Pi-hole’s `host` web port to 8081 doesn’t bother Caddy: it talks to `pihole:80` inside the network.

Step 4 — Add the `.home` names to Pi-hole’s DNS

Make the names resolve to the Pi. In the Pi-hole admin UI (<http://192.168.1.50:8081/admin> for now), go to **Settings** → **Local DNS Records** and add one **A record** per name, all pointing at the Pi’s **Tailscale** IP (`tailscale ip -4` on the Pi):

Domain	IP
<code>pihole.home</code>	<code>100.x.y.z</code>
<code>abs.home</code>	<code>100.x.y.z</code>

💡 Why the *Tailscale* IP, not the LAN IP

A DNS record holds one address. Pointing these at the Pi’s `100.x.y.z` tailnet address makes them reachable from *anywhere* a device is on your tailnet — home Wi-Fi or cellular alike. Caddy is listening on that address (Step 3), so the traffic lands. (This assumes Tailscale is running on the device, which is the premise of the whole series.)

Step 5 — Make Pi-hole your tailnet’s DNS

For `*.home` to resolve on every device, your devices must ask **Pi-hole** for DNS. In the Tailscale admin console (<https://login.tailscale.com>), **DNS** page:

1. **Add nameserver** → **Custom**, enter the Pi’s Tailscale IP (`100.x.y.z`), save.
2. Turn on **Override local DNS**. Now every tailnet device resolves through Pi-hole — so `*.home` names work, *and* you get Pi-hole ad-blocking on every device, even on cellular.

i Want only `.home` through Pi-hole, not all your DNS?

Use **split DNS** instead: in **Add nameserver** → **Custom**, enable **Restrict to domain** and set the domain to `home`. Then only `*.home` lookups go to Pi-hole and everything else uses each device's normal DNS. You lose tailnet-wide ad-blocking but keep the pretty URLs. ([Part 7](#) discusses how this DNS choice interacts with VPNs when you're away from home.)

Step 6 — Try it

From your laptop or phone, anywhere on the tailnet:

```
http://pihole.home    # Pi-hole admin (Caddy redirects to /admin)
http://abs.home       # Audiobookshelf
```

No ports, no IPs. To confirm resolution, ping `pihole.home` should answer from the Pi's `100.x.y.z`.

The pattern you'll reuse

Every service from here on gets a pretty URL the same way — three small steps:

1. Put its container on the `homelab` network.
2. Add a block to `~/proxy/Caddyfile`: `http://name.home { reverse_proxy container:PORT }`.
3. Add a `name.home` → `100.x.y.z` record in Pi-hole, then `docker compose restart caddy` (in `~/proxy`).

[Part 5](#) does exactly this for the dashboard, giving it `http://home.home`.

Caveats

- **Plain HTTP, no padlock.** `.home` has no public certificate authority, so these URLs are `http://`. On your authenticated tailnet that's fine. Real HTTPS needs a domain you own — point a subdomain (e.g. `pihole.example.com`) at the Pi and let Caddy fetch certificates automatically (the same Caddyfile with your real domain and the `http://` prefix removed).
- **Exit nodes break `.home`.** A Mullvad exit node routes DNS through Mullvad and bypasses Pi-hole, so `*.home` won't resolve while one is active. This tension is the subject of [Part 7](#); switch the exit node off (or use your home Pi as the exit node) to use the names.

Recap

- **Shared homelab network** so Caddy can reach services by name.
- **Pi-hole** moved its web UI to 8081 and now listens on all interfaces (LAN + tailnet).
- **Caddy** is the single front door on port 80, routing `pihole.home` and `abs.home` to the right containers.
- **Pi-hole DNS records + the Tailscale Override** make those names resolve on every device, everywhere.

You now have clean, portable URLs — and a repeatable recipe for every service to come. Next, [Part 5](#) adds a dashboard that ties them all together behind `http://home.home`.